

DISEÑO CONCEPTUAL Y LÓGICO

Ecommify

Plataforma e-commerce sobre el dataset Brazilian E-commerce (Olist)

Enfoque arquitectónico: Arquitectura políglota híbrida (PostgreSQL + MongoDB)

Equipo

Camilo José Mora Rodríguez

Carlos Alberto Zambrano Braydi

Miguel López Castañeda

Carlos Arturo Salazar Castañeda

Contexto del negocio

Ecommify es una plataforma e-commerce que opera sobre el dataset abierto Brazilian E-commerce (Olist). El diseño debe soportar el módulo transaccional (órdenes, pagos, envíos) y cargas analíticas sobre catálogo, reseñas y geolocalización.

9

tablas en esquema estrella
alrededor de orders

~1.0 M

filas en geolocation (volumen
dominante)

~99 K

órdenes / clientes / reviews

2016-18

ventana temporal (sept-2016
a oct-2018)

Hallazgos clave del EDA · integridad referencial intacta (0 huérfanos) · reviews con ~88 % sin título y ~59 % sin mensaje · fuerte concentración geográfica en SP, RJ y MG · ~97 % de pedidos en estado delivered

Requisitos y restricciones del dominio

Requisitos funcionales

- Registrar clientes, vendedores y catálogo de productos.
- Procesar órdenes con múltiples ítems, pagos y cuotas.
- Registrar estados logísticos (aprobado, envío, entrega).
- Capturar reseñas post-entrega y métricas de satisfacción.

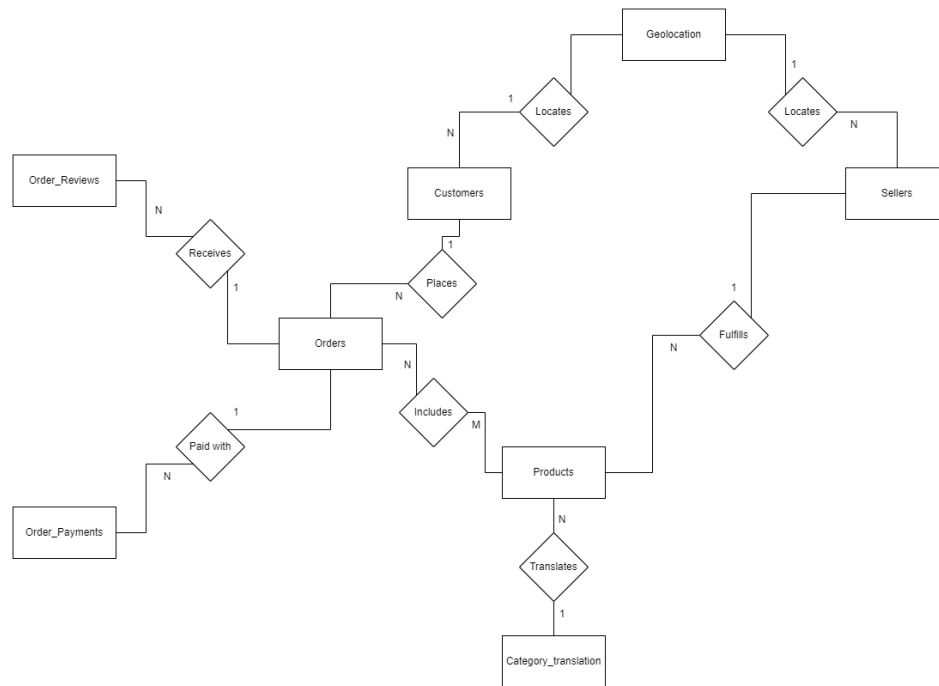
Requisitos no funcionales

- Consistencia fuerte (ACID) en órdenes y pagos.
- Disponibilidad y esquema flexible en catálogo y reviews.
- Soporte de cargas híbridas OLTP / OLAP.
- Operar dentro de límites del free tier (Supabase / Atlas).

Restricciones de negocio (CHECK / FK)

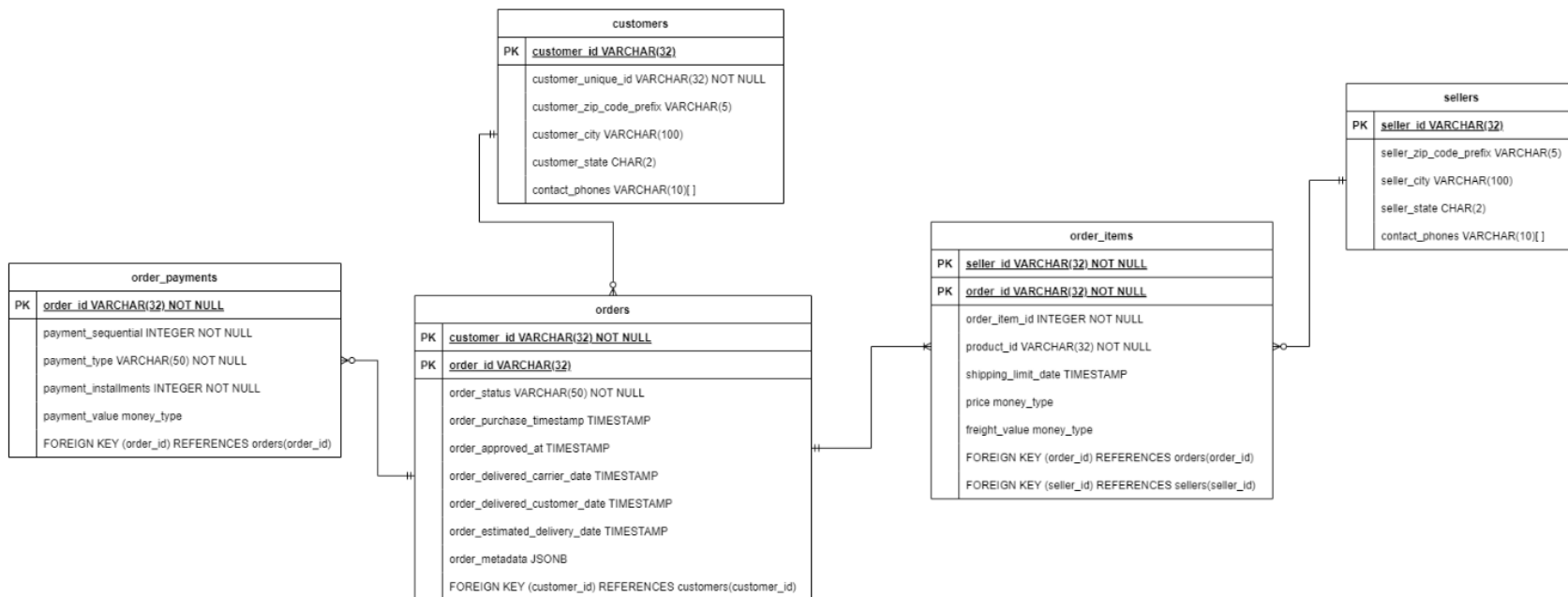
- **Valor monetario > 0**
price, freight_value y payment_value no admiten valores negativos.
- **Reseña $\in [1, 5]$**
review_score se valida con CHECK al insertar.
- **Cuotas ≥ 1**
payment_installments no puede ser cero ni negativo.
- **Dependencia orden $\leftrightarrow$ cliente**
Ninguna orden puede existir sin customer_id (FK NOT NULL).
- **Dependencia pago/reseña $\leftrightarrow$ orden**
FK obligatoria a orders; sin orden no hay pago ni review.

Modelo conceptual — Diagrama ER



Relaciones clave · Places (1:N) · Includes (N:M) · Fulfills (1:N) · Paid with (1:N) · Receives (1:N) · Locates (1:N)

Esquema relacional — Módulo PostgreSQL



Mínimo 3FN · PK explícitas · FK declaradas · tipos avanzados aplicados directo en el esquema: **order_metadata JSONB**, **contact_phones VARCHAR(10)[]**, **price/freight_value/payment_value money_type**.

Tipos avanzados PostgreSQL

JSONB

orders.order_metadata

Almacena la respuesta JSON cruda de la pasarela de pagos (resultado, mensaje, IDs externos). Evita una tabla auxiliar por proveedor y permite consultas con operadores @> y ->>.

ARRAY

customers.contact_phones · sellers.contact_phones

Cliente o vendedor con varios teléfonos sin tabla N:1 adicional. Reduce JOINS en consultas frecuentes de contacto.

COMPOSITE

order_items.price · order_items.freight_value · order_payments.payment_value

Tipo money_type (valor + moneda) para soportar pagos en BRL/USD u otras divisas sin romper integridad ni multiplicar columnas.

Extensiones PostgreSQL

pg_trgm — búsqueda tolerante a errores

Aplicación: customers.customer_city y sellers.seller_city.

Problema:

el dataset trae variaciones tipográficas — "sao paulo" vs "são paulo" — que producen falsos negativos en filtros y agregaciones.

Solución:

índice GIN con trigramas más operador % y similarity() para agrupar la misma ciudad escrita de varias formas.

Impacto:

mejora la calidad de los reportes geográficos sin tener que normalizar a mano un catálogo de ciudades.

PostGIS — a evaluar

Costos de envío por distancia
vendedor↔cliente y reportes geográficos
sobre los ~1 M de puntos de geolocation.

Otras consideradas

- **btree_gin**
índices GIN multi-tipo sobre JSONB + columnas escalares (búsqueda combinada en products).
- **hstore**
descartado a favor de JSONB.

Esquema MongoDB — Módulo analítico

6 colecciones (4.1)

Product_Catalog

catálogo rico con atributos dinámicos por categoría

order_reviews

calificaciones + texto libre + multimedia

recommendations

sugerencias ML personalizadas por cliente

Product_events

interacciones masivas en tiempo real

Notification

push, correo, estados de pedido

Analytics_events

tráfico, navegación, conversión, tiempos

Patrones de modelado (4.3)

Embedding · dimensiones físicas dentro del documento de producto.

Bucket · agrupar eventos continuos (views, carrito).

Reference · order_reviews guarda order_id (FK lógica a Postgres).

Computed · ventas y popularidad precalculadas para queries repetitivas.

Schema versioning · versiones de documento por evolución de catálogo.

Event · eventos en tiempo real para Kafka / pipelines.

Matriz de decisión por entidad

9 entidades · 6 criterios (Entidad / Tecnología / Naturaleza / CAP / Operaciones / Razón técnica) — 5.1

Entidad	Tec.	Naturaleza	CAP	Operaciones	Razón técnica
customers	PostgreSQL	Estructurado	C	INSERT, UPDATE, SELECT	Unicidad de ID, FK desde orders, datos personales con integridad.
sellers	PostgreSQL	Estructurado	C	INSERT, UPDATE, SELECT	Relaciones con productos y pagos; información fiscal.
orders	PostgreSQL	Estructurado	C	INSERT, UPDATE intensivo	Núcleo del negocio. ACID multi-tabla en un solo commit.
order_items	PostgreSQL	Estructurado	C	INSERT masivo	FKs compuestas hacia orders y products; conteos exactos.
order_payments	PostgreSQL	Estructurado	C	INSERT, UPDATE	Trazabilidad financiera; consistencia fuerte en dinero.
products	MongoDB	Semi-estructurado	A	find, aggregate masivo	Atributos variables por categoría; lectura masiva.
order_reviews	MongoDB	Semi-estructurado	A	find masivo, INSERT	Texto libre y calificaciones; escala horizontal de lectura.
geolocation	MongoDB	Semi-estructurado	A	Consultas geoespaciales	Índice 2dsphere; volumen alto, baja transaccionalidad.
user_behavior	MongoDB	Semi-estructurado	A	INSERT masivo	Esquema cambiante, append-only, sharding por fecha.

Análisis del Teorema CAP

Un sistema distribuido sólo garantiza dos de tres: Consistencia, Disponibilidad, Tolerancia a particiones. Ecommify asume P y reparte C/A por motor.

PostgreSQL — CP (Consistencia + Tolerancia a particiones)

Módulo transaccional.

Registrar un pago o actualizar el estado de una orden no puede arrojar datos desactualizados. Si un nodo falla, la operación espera o falla limpiamente antes de devolver información incorrecta.

Garantía:

modelo ACID + niveles de aislamiento configurables.

Aplica a:

customers, sellers, orders, order_items, order_payments.

MongoDB — AP (Disponibilidad + Tolerancia a particiones)

Módulo analítico y catálogo.

Consultar catálogo o reseñas puede tolerar segundos de desfase entre réplicas. No hay movimientos de dinero ni estados de órdenes activas en juego.

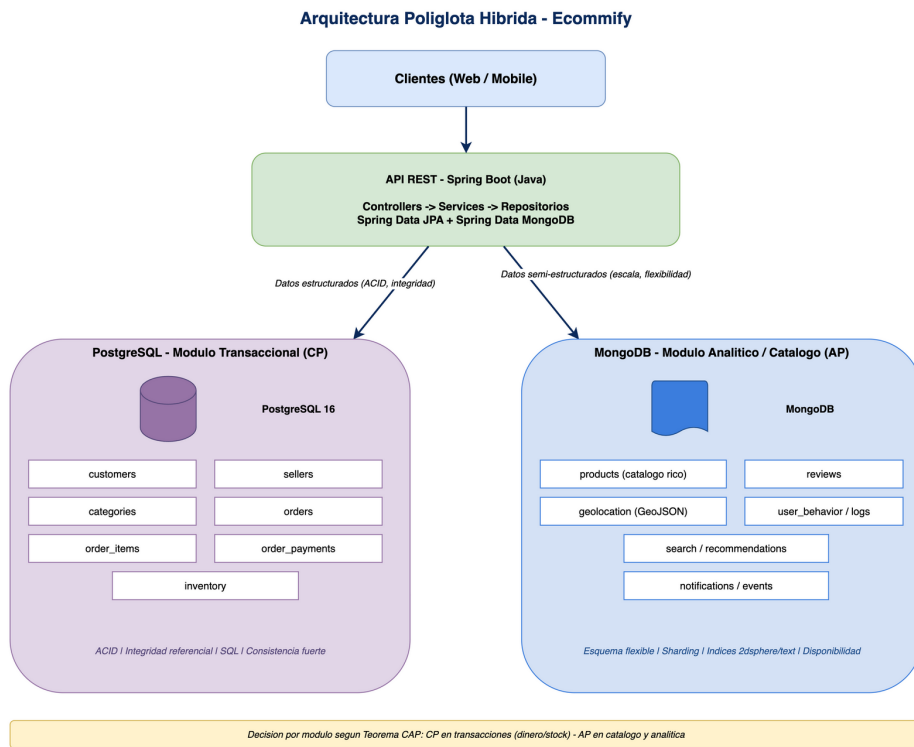
Garantía:

consistencia eventual sobre replica set.

Aplica a:

products, order_reviews, geolocation, user_behavior, recommendations.

Arquitectura híbrida políglota



PostgreSQL — CP

ACID, integridad referencial.
Tipos avanzados (JSONB, ARRAY, COMPOSITE).
Extensiones: pg_trgm (+ PostGIS a evaluar).

MongoDB — AP

Esquema flexible, sharding.
Índices 2dsphere para geolocation.
Índices text para búsqueda de productos y reviews.

Nota · el deck usa el esquema del 6 (Diccionario de datos / scripts SQL) alineado con Olist.

Sincronización Postgres → MongoDB

Ciertos identificadores existen en ambos motores. Esta es la estrategia que mantiene los datos coherentes — 5.4.

Dirección

Unidireccional

PostgreSQL es la fuente única de verdad. Mongo replica desde Postgres; no hay escrituras de vuelta hacia el OLTP.

Mecanismo

Job programado + updated_at

Un job consulta los cambios recientes en las tablas relevantes usando updated_at y aplica upserts en MongoDB.

Tolerancia a fallos

sync_failures + reintentos

Los errores se registran en la colección sync_failures con reintentos automáticos. Una caída de Mongo no afecta operaciones transaccionales.

Punto de control

sync_checkpoints

Tabla en Postgres con la última fecha de sincronización por entidad. Permite reanudar sin duplicar registros tras una caída.

Limitaciones y mitigaciones

Duplicación de identificadores (product_id, customer_id en ambos motores) 5.3

Mitigación · Costo aceptado por usar 2 bases. La aplicación mantiene coherencia vía el job de sincronización (\$5.4).

Doble modelo de acceso en el backend (Postgres + Mongo) 5.3

Mitigación · Separar responsabilidades por capa: cada repositorio habla con un único motor; servicios orquestan.

Consistencia eventual en el módulo analítico 5.3

Mitigación · Aceptable: una reseña tarda segundos en propagarse en el replica set; no afecta pagos ni órdenes activas.

Free tier de Supabase y MongoDB Atlas Operacional

Mitigación · Plan de escalamiento: paginar histórico, archivar particiones cold, mover blobs fuera de la BD.

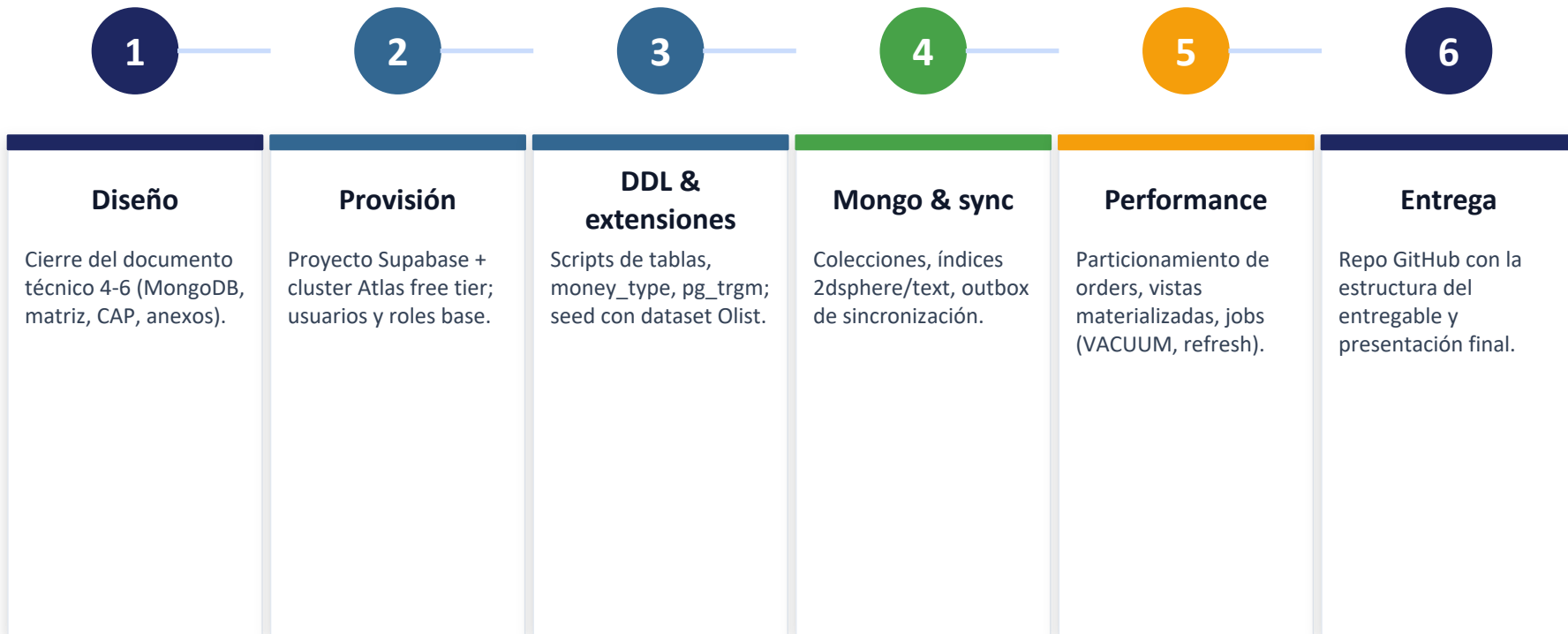
Geolocation con ~1 M filas y 261.831 duplicados por zip_code_prefix (N:N) EDA

Mitigación · Deduplicar en la carga inicial; índice 2dsphere en Mongo; agregación por prefijo cuando aplique.

Crecimiento de orders + consultas analíticas sobre la misma tabla OLTP Operacional

Mitigación · Particionamiento por fecha (hot/cold) + vistas materializadas (refresh semanal) + VACUUM diario.

Plan de implementación



Etapas siguientes: Entregable 2 — implementación física, índices y benchmarks.